

MATERIAL DE REVISÃO SEMESTRAL

Desenvolvimento Web Back-End

C# e .NET Core

Professor: Geovani

Instituição: Faculdade Einstein de Limeira

Ano: 2026

MATERIAL DE REVISÃO SEMESTRAL

Desenvolvimento Web Back-End - C# e .NET

Professor: Geovani

Instituição: Faculdade Einstein de Limeira

Disciplina: Desenvolvimento Web Back-End

Ano: 2026

SUMÁRIO

1. [.NET Core e Multiplataforma](#)
 2. [Fundamentos de C# - Parte 1](#)
 3. [Fundamentos de C# - Parte 2](#)
 4. [LINQ - Language Integrated Query](#)
 5. [Programação Orientada a Objetos - Parte 1](#)
 6. [Programação Orientada a Objetos - Parte 2](#)
 7. [Resumo Final - Pontos-Chave](#)
-

1. .NET CORE E MULTIPLATAFORMA

1.1 Evolução da Plataforma .NET

Linha do Tempo

- 2002 → .NET Framework 1.0
 - └ Apenas Windows
 - └ Monolítico
 - └ Dependente do sistema operacional

- 2016 → .NET Core 1.0
 - └ Cross-platform (Windows, Linux, macOS)
 - └ Open-source
 - └ Modular e leve
 - └ CoreCLR (runtime reescrito)

- 2020 → .NET 5 (Unificação)
 - └ .NET Framework + .NET Core = .NET 5+
 - └ Versão única multiplataforma
 - └ Sem o nome "Core"

- 2024 → .NET 8 (LTS - Long Term Support)
 - └ Suporte estendido (3 anos)
 - └ Performance otimizada
 - └ ☒ VERSÃO ATUAL DO CURSO

1.2 CoreCLR - A Chave da Multiplataforma

O que é CoreCLR?

CoreCLR = **C**ore **C**ommon Language **R**untime

- ☒ **Reescrito do zero** (não é adaptação do CLR do .NET Framework)
- ☒ **Removeu dependências do Windows**
- ☒ **Arquitetura modular e leve**
- ☒ **Compatível com Linux e macOS**

Como Funciona a Execução Multiplataforma?

```
Código C# (.cs)
    ↓
Compilador Roslyn
    ↓
IL Code (Intermediate Language) ← Código intermediário PORTÁVEL
    ↓
JIT Compiler (Just-In-Time) ← Converte IL para código NATIVO
    ↓
Execução no Sistema Operacional (Windows/Linux/macOS)
```

⚙️ **JIT Compiler:** - Traduz IL para código **nativo da máquina** em **tempo de execução** - Adapta-se ao sistema operacional e hardware - Por isso C# roda em qualquer plataforma!

Diferenças Principais

Característica	.NET Framework	.NET Core / .NET 5+
Plataforma	✗ Apenas Windows	✓ Cross-platform
Runtime	CLR (legado)	CoreCLR (reescrito)
Open Source	✗ Não	✓ Sim
Arquitetura	Monolítica	Modular
Dependências Windows	✓ Sim	✗ Não
Futuro	Manutenção apenas	✓ Ativo

1.3 Pontos-Chave para ENADE

- ✓ **CoreCLR** foi a chave para multiplataforma (não só o JIT)
- ✓ .NET 5+ é a **unificação**, não voltou a ser Windows-only
- ✓ C# 6.0 **NÃO** introduziu OOP (OOP existe desde C# 1.0)
- ✓ Projetos **Console, Web API, MVC** podem ser multiplataforma
- ✓ O comando `dotnet new console` cria projeto multiplataforma, mas **outros tipos também são**

2. FUNDAMENTOS DE C# - PARTE 1

2.1 Nullable Value Types (Tipos Anuláveis)

Conceito

Por padrão, **tipos de valor** (int, double, bool, etc) **NÃO podem ser null**.

```
// ❌ ERRO: não compila
int numero = null;

// ✅ CORRETO: usar ? para tornar anulável
int? numero = null;
```

Sintaxe

```
// Declaração de nullable
int? idade = null;
double? altura = 1.75;
bool? ativo = true;

// Verificando se tem valor
if (idade.HasValue)
{
    Console.WriteLine($"Idade: {idade.Value}");
}

// Operador ?? (null-coalescing)
int idadeReal = idade ?? 0; // Se null, usa 0
```

Por que usar Nullable?

- ✅ Campos opcionais em formulários (idade, telefone)
- ✅ Integração com banco de dados (campos NULL)
- ✅ Representar "não informado" vs "zero"

Tabela Resumo

Tipo	Pode ser null?	Sintaxe
int	❌ Não	int numero = 10;
int?	✅ Sim	int? numero = null;
string	✅ Sim (é tipo de referência)	string nome = null;
bool	❌ Não	bool ativo = true;
bool?	✅ Sim	bool? ativo = null;

2.2 Controle de Fluxo - break vs continue

break - Encerra o Loop

```
for (int i = 0; i < 10; i++)
{
    if (i == 5)
        break; // ❌ SAI do loop IMEDIATAMENTE

    Console.WriteLine(i);
}
// Saída: 0, 1, 2, 3, 4
```

continue - Pula para Próxima Iteração

```
for (int i = 0; i < 10; i++)
{
    if (i == 5)
        continue; // 🔄 PULA apenas esta iteração

    Console.WriteLine(i);
}
// Saída: 0, 1, 2, 3, 4, 6, 7, 8, 9
```

Tabela Comparativa

Comando	O que faz	Após execução
break	Encerra o loop completamente	Vai para DEPOIS do loop
continue	Pula para próxima iteração	Volta para INÍCIO do loop
return	Sai da função/método	Retorna para quem chamou

2.3 Operador Ternário

Sintaxe

```
tipo variavel = (condição) ? valorSeVerdadeiro : valorSeFalso;
```

Exemplos

```
// Exemplo 1: Atribuição
int idade = 17;
string status = (idade >= 18) ? "Maior de idade" : "Menor de idade";

// Exemplo 2: Return
public string ObterStatus(int nota)
{
    return (nota >= 6) ? "Aprovado" : "Reprovado";
}

// Exemplo 3: Aninhado (evitar!)
string resultado = (nota >= 9) ? "Excelente"
                    : (nota >= 7) ? "Bom"
                    : (nota >= 5) ? "Regular"
                    : "Insuficiente";
```

Equivalência com if/else

```
// Com operador ternário
string resultado = (x > 5) ? "Maior" : "Menor";

// Com if/else (equivalente)
string resultado;
if (x > 5)
    resultado = "Maior";
else
    resultado = "Menor";
```

Quando usar?

✓ **USE:** Atribuições simples e condições diretas

✗ **EVITE:** Lógica complexa, múltiplos níveis, operações com efeitos colaterais

2.4 Afirmações - Resumo

Questão ENADE: Analise as afirmações

I. `int? numero = null;` está correto, pois `?` torna o tipo anulável. ✓ **VERDADEIRO** - Nullable value type

II. `continue` encerra o loop imediatamente. ✗ **FALSO** - `continue` pula para próxima iteração, `break` que encerra

III. Operador ternário é forma compacta de if/else. ✓ **VERDADEIRO** - Substitui if/else simples

Resposta: Apenas I e III estão corretas.

3. FUNDAMENTOS DE C# - PARTE 2

3.1 Coleções Genéricas

Visão Geral

Coleção	Tipo	Tamanho	Características
Array	<code>int[]</code>	✗ Fixo	Acesso rápido por índice
List<T>	<code>List<int></code>	✓ Dinâmico	Add(), Remove(), tamanho variável
Dictionary<K,V>	<code>Dictionary<int, string></code>	✓ Dinâmico	Chave-Valor, busca O(1)
HashSet<T>	<code>HashSet<int></code>	✓ Dinâmico	Sem duplicatas, busca rápida
Queue<T>	<code>Queue<int></code>	✓ Dinâmico	FIFO (First-In, First-Out)
Stack<T>	<code>Stack<int></code>	✓ Dinâmico	LIFO (Last-In, First-Out)

3.2 List<T> - A Mais Usada

Por que usar List<T>?

- ✓ **Tamanho DINÂMICO** - cresce e diminui automaticamente
- ✓ **Type-safe** - genérico, evita casting
- ✓ **Métodos úteis** - Add, Remove, Find, Sort, etc
- ✓ **Performance** - acesso por índice O(1)

Sintaxe e Métodos

```
// Criar
List<string> nomes = new List<string>();

// Adicionar
nomes.Add("João");
nomes.Add("Maria");
nomes.Add("Pedro");

// Acessar por índice
string primeiro = nomes[0]; // "João"

// Remover
nomes.RemoveAt(1); // Remove "Maria" (índice 1)
nomes.Remove("Pedro"); // Remove por valor

// Verificar
bool existe = nomes.Contains("João"); // true
int quantidade = nomes.Count; // 1

// Buscar
string encontrado = nomes.Find(n => n.StartsWith("J")); // "João"

// Ordenar
nomes.Sort(); // Ordem alfabética
```

List\<T> vs Array

```
// ❌ Array - tamanho FIXO
int[] numeros = new int[5]; // Sempre 5 elementos
numeros[0] = 10;
numeros[1] = 20;
// não tem Add() ou Remove()

// ✅ List<T> - tamanho DINÂMICO
List<int> numeros = new List<int>();
numeros.Add(10); // Adiciona
numeros.Add(20); // Adiciona
numeros.Add(30); // Adiciona quantos quiser!
numeros.RemoveAt(0); // Remove quando quiser
```

3.3 Outras Coleções - Resumo

Dictionary\<K, V> - Chave-Valor

```
Dictionary<int, string> alunos = new Dictionary<int, string>();
alunos.Add(1, "João");
alunos.Add(2, "Maria");

string nome = alunos[1]; // "João"
bool existe = alunos.ContainsKey(2); // true
```

Características: -  Busca por chave O(1) - muito rápida -  Chaves únicas (não permite duplicatas) -  Não permite chaves duplicadas

HashSet\<T> - Sem Duplicatas

```
HashSet<int> numeros = new HashSet<int>();
numeros.Add(1);
numeros.Add(2);
numeros.Add(1); // Ignora! Não adiciona duplicata

Console.WriteLine(numeros.Count); // 2
```

Características: -  Garante elementos únicos -  Busca rápida O(1) -  **NÃO** é chave-valor -  Sem acesso por índice

Queue\<T> - Fila (FIFO)

```
Queue<string> fila = new Queue<string>();
fila.Enqueue("Primeiro"); // Adiciona no fim
fila.Enqueue("Segundo");
fila.Enqueue("Terceiro");

string atendido = fila.Dequeue(); // "Primeiro" (remove do início)
```

FIFO = First-In, First-Out (primeiro que entra, primeiro que sai)

Stack\<T> - Pilha (LIFO)

```
Stack<string> pilha = new Stack<string>();
pilha.Push("Primeiro"); // Adiciona no topo
pilha.Push("Segundo");
pilha.Push("Terceiro");

string removido = pilha.Pop(); // "Terceiro" (remove do topo)
```

LIFO = Last-In, First-Out (último que entra, primeiro que sai)

3.4 Pontos-Chave para ENADE

- ✓ **List<T>** é a resposta quando pergunta sobre **tamanho dinâmico**
- ✗ Dictionary **NÃO** garante ausência de duplicatas (HashSet garante)
- ✗ Queue é **FIFO**, não LIFO
- ✗ HashSet **NÃO** é chave-valor (Dictionary é)
- ✓ Array Multidimensional tem tamanho **FIXO**, não dinâmico

4. LINQ - LANGUAGE INTEGRATED QUERY

4.1 O que é LINQ?

LINQ = Language **I**ntegrated **Q**uery

- Consultas integradas à linguagem C#
- Trabalha com coleções (List, Array, etc)
- Sintaxe declarativa (diz O QUE quer, não COMO fazer)

4.2 Métodos Principais

Where - Filtrar

Propósito: Filtrar elementos que atendem uma condição

```
List<int> numeros = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

// Filtrar apenas números maiores que 5
var maioresQue5 = numeros.Where(n => n > 5);
// Resultado: { 6, 7, 8, 9, 10 }

// Filtrar pares
var pares = numeros.Where(n => n % 2 == 0);
// Resultado: { 2, 4, 6, 8, 10 }
```

Select - Transformar

Propósito: Transformar cada elemento (projeção)

```
List<int> numeros = new List<int> { 1, 2, 3, 4, 5 };

// Multiplicar cada número por 2
var dobrados = numeros.Select(n => n * 2);
// Resultado: { 2, 4, 6, 8, 10 }

// Transformar en string
var textos = numeros.Select(n => $"Número {n}");
// Resultado: { "Número 1", "Número 2", ... }
```

Encadeamento (Chaining)

```
List<int> numeros = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

var resultado = numeros
    .Where(n => n > 5)      // 1º: Filtrar > 5 → { 6, 7, 8, 9, 10 }
    .Select(n => n * 2)     // 2º: Dobrar cada → { 12, 14, 16, 18, 20 }
    .ToList();             // 3º: Converter para List

// Resultado final: { 12, 14, 16, 18, 20 }
```

4.3 Outros Métodos Úteis

```
List<int> numeros = new List<int> { 5, 2, 8, 1, 9, 3 };

// OrderBy - Ordenar crescente
var ordenados = numeros.OrderBy(n => n); // { 1, 2, 3, 5, 8, 9 }

// OrderByDescending - Ordenar decrescente
var desc = numeros.OrderByDescending(n => n); // { 9, 8, 5, 3, 2, 1 }

// FirstOrDefault - Primeiro ou valor padrão
int primeiro = numeros.FirstOrDefault(n => n > 10); // 0 (nenhum > 10)

// Any - Verifica se existe algum
bool existeMaior = numeros.Any(n => n > 7); // true

// All - Verifica se todos atendem
bool todosPositivos = numeros.All(n => n > 0); // true

// Count - Conta elementos
int quantos = numeros.Count(n => n > 5); // 3

// Sum, Average, Min, Max
int soma = numeros.Sum(); // 28
double media = numeros.Average(); // 4.67
int minimo = numeros.Min(); // 1
int maximo = numeros.Max(); // 9
```

4.4 Tabela Resumo

Método	Propósito	Retorno	Exemplo
Where	Filtrar	IEnumerable	<code>Where(n => n > 5)</code>
Select	Transformar	IEnumerable	<code>Select(n => n * 2)</code>
OrderBy	Ordenar crescente	IOrderedEnumerable	<code>OrderBy(n => n)</code>
First	Primeiro elemento	T	<code>First(n => n > 5)</code>
Any	Existe algum?	bool	<code>Any(n => n > 10)</code>
All	Todos atendem?	bool	<code>All(n => n > 0)</code>
Count	Contar	int	<code>Count(n => n > 5)</code>
Sum	Somar	int/decimal	<code>Sum()</code>

4.5 Pontos-Chave para ENADE

- ✓ **Where** filtra, **Select** transforma
- ✗ Where **NÃO** agrupa (GroupBy agrupa)
- ✗ Select **NÃO** ordena (OrderBy ordena)
- ✓ Resultado de `Where(n => n > 5).Select(n => n * 2)` em `{1..10}` é **{12, 14, 16, 18, 20}**
- ✗ LINQ **NÃO** faz contagem automática (Count() faz)

5. PROGRAMAÇÃO ORIENTADA A OBJETOS - PARTE 1

5.1 Encapsulamento - Fields vs Properties

Conceito

Encapsulamento = Ocultar estado interno, expor apenas o necessário

- **Field (campo):** Variável privada que armazena dados
- **Property (propriedade):** Interface pública com get/set

Sintaxe Completa

```
public class Produto
{
    // FIELD - privado, armazena o valor
    private decimal preco;

    // PROPERTY - público, controla acesso
    public decimal Preco
    {
        get { return preco; } // Leitura
        set // Escrita
        {
            if (value < 0)
                throw new ArgumentException("Preço não pode ser negativo!");

            preco = value; // ✓ ATRIBUIÇÃO CORRETA
        }
    }
}
```

Auto-Properties (Simplificado)

```
// Quando NÃO precisa de validação
public class Produto
{
    public string Nome { get; set; } // Compiler cria field automaticamente
    public decimal Preco { get; set; }
}
```

Property com Validação

```
public class Produto
{
    private decimal preco;

    public decimal Preco
    {
        get => preco; // Sintaxe moderna (expression-bodied)
        set
        {
            if (value < 0)
                throw new ArgumentException("Preço negativo!");

            preco = value; // ✅ Atribui ao FIELD
        }
    }
}
```

Read-Only Property

```
public class Produto
{
    public int Id { get; private set; } // Apenas leitura externa

    public Produto(int id)
    {
        Id = id; // Pode setar no construtor
    }
}
```

5.2 Palavra-chave `value`

No **setter**, `value` é uma variável implícita que contém o valor sendo atribuído.

```

public decimal Preco
{
    set
    {
        // 'value' = valor que está sendo atribuído
        if (value < 0)
            throw new ArgumentException("Preço negativo!");

        preco = value; // Atribui 'value' ao field 'preco'
    }
}

// Uso:
produto.Preco = 100; // 'value' = 100

```

5.3 Membros Estáticos (static)

Conceito

static = Pertence à CLASSE, não ao objeto

```

public class Contador
{
    // ✅ ESTÁTICO - compartilhado por todos os objetos
    public static int Total = 0;

    // ❌ NÃO ESTÁTICO - cada objeto tem o seu
    public int MeuNumero;

    public Contador()
    {
        Total++; // Incrementa o contador global
        MeuNumero = Total;
    }
}

// Uso:
Contador c1 = new Contador();
Contador c2 = new Contador();
Contador c3 = new Contador();

Console.WriteLine(Contador.Total); // 3 (acessado pela CLASSE)
Console.WriteLine(c1.MeuNumero); // 1 (acessado pelo OBJETO)
Console.WriteLine(c2.MeuNumero); // 2
Console.WriteLine(c3.MeuNumero); // 3

```


Restrições de Membros Estáticos

- ✗ **NÃO pode** usar `this` (`this` = instância atual)
- ✗ **NÃO pode** acessar campos de instância diretamente
- ✓ **Pode** acessar apenas outros membros estáticos

```
public class Exemplo
{
    private int numero = 10;           // Campo de instância
    private static int total = 0;      // Campo estático

    // ✓ Método de instância - pode acessar TUDO
    public void MetodoInstancia()
    {
        Console.WriteLine(numero);    // ✓ OK
        Console.WriteLine(total);     // ✓ OK
        Console.WriteLine(this.numero); // ✓ OK
    }

    // ✗ Método estático - acessa apenas estáticos
    public static void MetodoEstatico()
    {
        Console.WriteLine(numero);    // ✗ ERRO! não pode acessar instância
        Console.WriteLine(total);     // ✓ OK
        Console.WriteLine(this.numero); // ✗ ERRO! não tem 'this'
    }
}
```

Classes Estáticas

Classes utilitárias que **NUNCA** serão instanciadas.

```

public static class Calculadora
{
    // ✅ Todos os membros DEVEM ser static
    public static int Somar(int a, int b)
    {
        return a + b;
    }

    public static double Dividir(double a, double b)
    {
        return a / b;
    }
}

// Uso:
int resultado = Calculadora.Somar(10, 5); // 15

// ❌ ERRO: não pode instanciar
// Calculadora calc = new Calculadora();

```

Exemplos do Framework

```

Math.Pow(2, 3);           // Math é classe estática
Console.WriteLine("Oi"); // Console é classe estática
Convert.ToInt32("10");    // Convert é classe estática

```

5.4 Afirmações sobre static

I. Método estático pode acessar campos de instância se forem public. ❌ FALSO - Métodos estáticos **NÃO** acessam campos de instância (nem public)

II. `this` é proibido em membros estáticos. ✅ VERDADEIRO - `this` refere-se à instância, estáticos não têm instância

III. Membros estáticos são compartilhados por todas as instâncias. ✅ VERDADEIRO - Existe uma única cópia na memória

IV. Classes estáticas exigem que todos os membros sejam static. ✅ VERDADEIRO - Não faz sentido ter membros de instância em classe estática

Resposta: II, III e IV estão corretas.

5.5 Sobrecarga de Construtores

Conceito

Múltiplos construtores com **diferentes parâmetros**.

```
public class Produto
{
    public string Nome { get; set; }
    public decimal Preco { get; set; }

    // Construtor 1 - Completo
    public Produto(string nome, decimal preco)
    {
        Nome = nome;
        Preco = preco;
    }

    // Construtor 2 - Apenas nome (Preco = 0)
    public Produto(string nome)
    {
        Nome = nome;
        Preco = 0;
    }

    // Construtor 3 - Sem parâmetros
    public Produto()
    {
        Nome = "Sem nome";
        Preco = 0;
    }
}

// Uso:
Produto p1 = new Produto("Notebook", 3000); // Construtor 1
Produto p2 = new Produto("Mouse");          // Construtor 2
Produto p3 = new Produto();                  // Construtor 3
```

Encadeamento com `this`

Evitar duplicação de código chamando outro construtor.

```

public class Produto
{
    public string Nome { get; set; }
    public decimal Preco { get; set; }

    // Construtor principal (mais completo)
    public Produto(string nome, decimal preco)
    {
        Nome = nome;
        Preco = preco;
    }

    // Construtor que chama o principal
    public Produto(string nome) : this(nome, 0)
    {
        // 'this(nome, 0)' chama Produto(string, decimal)
        // Não precisa duplicar a lógica de Nome e Preco
    }

    // Outro construtor que chama o principal
    public Produto() : this("Sem nome", 0)
    {
    }
}

```

Sintaxe `this` VS `base`

```

// this(...) - chama OUTRO CONSTRUTOR da MESMA CLASSE
public Produto(string nome) : this(nome, 0)
{
}

// base(...) - chama construtor da CLASSE BASE (herança)
public Aluno(string nome) : base(nome)
{
}

```

5.6 Pontos-Chave para ENADE

- ✓ Para atribuir ao field no setter: `preco = value;`
- ✗ **NÃO** usar `this.Preco = value;` (loop infinito!)
- ✗ **NÃO** usar `return preco;` (return é só no get)
- ✓ Encadeamento: `: this(nome, 0)`
- ✗ **NÃO** é `this.Produto(nome, 0);`
- ✗ **NÃO** é `new Produto(nome, 0);`

6. PROGRAMAÇÃO ORIENTADA A OBJETOS - PARTE 2

6.1 Herança (Inheritance)

Conceito

Mecanismo onde uma classe **herda** características de outra.

```
Classe Base (ou Superclasse ou Pai)
    ↓
herda
    ↓
Classe Derivada (ou Subclasse ou Filha)
```

Objetivos da Herança

1.  **Reutilização de código** - evitar duplicação
2.  **Relação "É UM(A)"** - Aluno **é uma** Pessoa
3.  **Hierarquia de classes** - organização lógica
4.  **Polimorfismo** - tratar objetos de forma genérica

Sintaxe

```
// Classe Base
public class Pessoa
{
    public string Nome { get; set; }
    public int Idade { get; set; }

    public void ApresentarSe()
    {
        Console.WriteLine($"Olá, sou {Nome}");
    }
}

// Classe Derivada
public class Aluno : Pessoa // "Aluno herda de Pessoa"
{
    public string Matricula { get; set; }
    public double Nota { get; set; }

    // Aluno tem: Nome, Idade (herdados) + Matricula, Nota (próprios)
}

// Uso:
Aluno aluno = new Aluno();
aluno.Nome = "João";      // ✔ Herdado de Pessoa
aluno.Idade = 20;         // ✔ Herdado de Pessoa
aluno.Matricula = "12345"; // ✔ Próprio de Aluno
aluno.ApresentarSe();    // ✔ Método herdado
```

Palavra-chave **base**

```
public class Pessoa
{
    public string Nome { get; set; }

    public Pessoa(string nome)
    {
        Nome = nome;
    }
}

public class Aluno : Pessoa
{
    public string Matricula { get; set; }

    // Construtor chama construtor da base
    public Aluno(string nome, string matricula) : base(nome)
    {
        Matricula = matricula;
    }
}
```

Ordem de Execução de Construtores

1. Construtor da CLASSE BASE (Pessoa)
2. Construtor da CLASSE DERIVADA (Aluno)

 **FALSO:** "Classe Derivada executa antes da Base"

 **VERDADEIRO:** "Classe Base executa ANTES da Derivada"

6.2 Classes Abstratas

Conceito

Classes que servem apenas como **base**, não podem ser instanciadas.

```

public abstract class Forma // 'abstract' = não pode instanciar
{
    // Método abstrato - SEM implementação (apenas assinatura)
    public abstract double CalcularArea();

    // Método concreto - COM implementação
    public void Exibir()
    {
        Console.WriteLine($"Área: {CalcularArea()}");
    }
}

public class Circulo : Forma
{
    public double Raio { get; set; }

    // OBRIGATÓRIO implementar métodos abstratos
    public override double CalcularArea()
    {
        return Math.PI * Raio * Raio;
    }
}

public class Retangulo : Forma
{
    public double Largura { get; set; }
    public double Altura { get; set; }

    public override double CalcularArea()
    {
        return Largura * Altura;
    }
}

// Uso:
Forma forma = new Forma(); // ❌ ERRO! Classe abstrata
Forma circulo = new Circulo(); // ✅ OK! Instancia a derivada
circulo.Exibir();

```

Características

- ✅ **Pode** ter métodos abstratos (sem corpo)
- ✅ **Pode** ter métodos concretos (com corpo)
- ✅ **Pode** ter construtores
- ✅ **Pode** ter propriedades e campos
- ❌ **NÃO pode** ser instanciada com `new`
- ✅ **Serve** como classe base para outras

Principal Restrição

✗ Não pode ser instanciada diretamente com `new`

```
Forma f = new Forma(); // ✗ ERRO: Cannot create instance of abstract class
```

6.3 Interfaces

Conceito

Contrato que define **O QUE** uma classe deve fazer, sem dizer **COMO**.

```
// Interface - define contrato
public interface INotificador
{
    void Enviar(string mensagem); // Apenas assinatura, sem corpo
}

// Implementação 1
public class EmailNotificador : INotificador
{
    public void Enviar(string mensagem)
    {
        Console.WriteLine($"✉ Enviando email: {mensagem}");
    }
}

// Implementação 2
public class SmsNotificador : INotificador
{
    public void Enviar(string mensagem)
    {
        Console.WriteLine($"📱 Enviando SMS: {mensagem}");
    }
}

// Uso polimórfico:
INotificador notificador = new EmailNotificador();
notificador.Enviar("Olá!"); // ✉ Enviando email: Olá!

notificador = new SmsNotificador();
notificador.Enviar("Olá!"); // 📱 Enviando SMS: Olá!
```

Implementação Múltipla

Diferença principal entre Interface e Classe Base:

```
//  PODE implementar MÚLTIPLAS interfaces
public class EmailSms : INotificador, ILogger, IConfiguravel
{
    public void Enviar(string msg) { }
    public void Log(string msg) { }
    public void Configurar() { }
}

//  NÃO PODE herdar de múltiplas classes
public class Aluno : Pessoa, Funcionario //  ERRO!
{
}
```

Interface vs Classe Abstrata

Característica	Interface	Classe Abstrata
Implementação múltipla	 Sim (várias interfaces)	 Não (apenas 1 classe base)
Métodos com corpo	 Não (apenas assinaturas)	 Sim (pode ter concretos)
Campos	 Não	 Sim
Construtores	 Não	 Sim
Herança	Implementação	Herança
Palavra-chave	interface	abstract class
Uso	Contrato de comportamento	Base compartilhada

Nomenclatura de Interfaces

 **Convenção:** Iniciar com **I** maiúsculo

```
INotificador
IRepository
ILogger
IConfiguravel
```

6.4 Pontos-Chave para ENADE

Herança

-  **Objetivo:** Reutilização, relação "É UM(A)", hierarquia
-  **NÃO é:** Definir contrato (isso é Interface)
-  **NÃO é:** Impedir acesso ao estado interno (isso é Encapsulamento)
-  **NÃO é:** Múltiplas implementações (isso é Polimorfismo)

Classes Abstratas

- ✓ **Restrição principal:** Não pode ser instanciada
- ✗ **Pode sim:** Ter construtores
- ✗ **Pode sim:** Ter métodos concretos
- ✗ **Pode sim:** Ter propriedades e campos
- ✗ **Pode sim:** Servir como classe base

Interfaces

- ✓ **Diferencial:** Implementação múltipla (várias interfaces simultaneamente)
- ✗ **NÃO pode:** Ter campos
- ✗ **NÃO pode:** Ter construtores
- ✗ **NÃO pode:** Definir comportamento padrão (apenas assinaturas)
- ✗ **NÃO tem:** `sealed` (`sealed` é para classes)

7. RESUMO FINAL - PONTOS-CHAVE

7.1 Checklist ENADE

.NET Core

- [] CoreCLR foi reescrito do zero para ser multiplataforma
- [] .NET 5+ é a unificação, NÃO voltou a ser Windows-only
- [] JIT traduz IL para código nativo em runtime
- [] Qualquer tipo de projeto pode ser multiplataforma (Console, Web, etc)

Fundamentos C

- [] `int?` torna tipo de valor anulável (nullable)
- [] `continue` pula iteração, `break` encerra loop
- [] Operador ternário: `(cond) ? verdadeiro : falso`
- [] `List<T>` tem tamanho dinâmico (Add, Remove)
- [] Array tem tamanho fixo

LINQ

- [] `Where` filtra elementos
- [] `Select` transforma elementos
- [] Encadeamento: `Where().Select().ToList()`
- [] `Where(n => n > 5)` retém apenas maiores que 5
- [] `Select(n => n * 2)` dobra cada elemento

OOP - Encapsulamento

- [] Field privado armazena, Property pública expõe
- [] No setter, `value` é o valor sendo atribuído
- [] Atribuir: `campo = value;` (não `this.Propriedade = value`)
- [] Auto-property: `public string Nome { get; set; }`

OOP - static

- [] `static` pertence à classe, não ao objeto
- [] Método estático NÃO acessa campos de instância
- [] Método estático NÃO usa `this`
- [] Classe estática: todos membros devem ser static
- [] Membros estáticos são compartilhados por todas instâncias

OOP - Construtores

- [] Sobrecarga = múltiplos construtores com parâmetros diferentes
- [] Encadeamento: `: this(param1, param2)`
- [] `base` chama construtor da classe base
- [] Ordem: Base → Derivada

OOP - Herança

- [] Objetivo: reutilização, relação "É UM(A)"
- [] Sintaxe: `class Aluno : Pessoa`
- [] `base` acessa membros da classe base
- [] Construtor base executa ANTES da derivada

OOP - Classes Abstratas

- [] `abstract class` não pode ser instanciada
- [] Pode ter métodos abstratos (sem corpo)
- [] Pode ter métodos concretos (com corpo)
- [] Pode ter construtores, campos, propriedades
- [] Serve como base para outras classes

OOP - Interfaces

- [] Define contrato (O QUE fazer)
- [] Apenas assinaturas, sem implementação
- [] Implementação múltipla: `class X : IA, IB, IC`
- [] Nomenclatura: inicia com `I` maiúsculo
- [] NÃO tem campos, NÃO tem construtores

7.2 Armadilhas Comuns ENADE

- ✗ **C# 6.0 introduziu OOP** → OOP existe desde C# 1.0
- ✗ **continue encerra o loop** → break encerra, continue pula
- ✗ **Dictionary garante sem duplicatas** → HashSet garante
- ✗ **Queue é LIFO** → Queue é FIFO, Stack é LIFO
- ✗ **Where agrupa dados** → GroupBy agrupa
- ✗ **Select ordena** → OrderBy ordena
- ✗ **Setter:** `return preco;` → return só no get
- ✗ **Setter:** `this.Preco = value;` → loop infinito!
- ✗ **Encadeamento:** `new Produto()` → usar `: this()`
- ✗ **Método estático acessa campo de instância** → NÃO pode
- ✗ **Classe derivada constrói antes da base** → base primeiro
- ✗ **Classe abstrata não pode ter métodos concretos** → pode sim
- ✗ **Interface permite múltiplas implementações** → isso é diferencial!

7.3 Tabela Rápida - Quando Usar O Quê?

Situação	Use
Tamanho dinâmico, Add/Remove	<code>List<T></code>
Chave-Valor, busca rápida	<code>Dictionary<K,V></code>
Sem duplicatas	<code>HashSet<T></code>
Fila (FIFO)	<code>Queue<T></code>
Pilha (LIFO)	<code>Stack<T></code>
Filtrar coleção	<code>Where</code>
Transformar elementos	<code>Select</code>
Campo pode ser null	<code>int? (nullable)</code>
Pular iteração	<code>continue</code>
Encerrar loop	<code>break</code>
If/else simples	Operador ternário
Validação no setter	Property com field privado
Compartilhar entre objetos	<code>static</code>
Chamar outro construtor	<code>: this()</code>
Reutilizar código, "É UM(A)"	Herança
Base para outras classes	Classe abstrata
Contrato de comportamento	Interface
Implementar várias "coisas"	Múltiplas interfaces

7.4 Código Rápido - Referências

Nullable

```
int? idade = null;  
if (idade.HasValue)  
    Console.WriteLine(idade.Value);
```

Continue vs Break

```
for (int i = 0; i < 5; i++)  
{  
    if (i == 2) continue; // pula 2  
    if (i == 4) break;    // para em 4  
    Console.WriteLine(i); // 0, 1, 3  
}
```

Ternário

```
string status = (nota >= 6) ? "Aprovado" : "Reprovado";
```

List

```
List<int> nums = new List<int>();  
nums.Add(10);  
nums.Remove(10);
```

LINQ

```
var resultado = nums  
    .Where(n => n > 5)  
    .Select(n => n * 2)  
    .ToList();
```

Property

```
private decimal preco;
public decimal Preco
{
    get => preco;
    set
    {
        if (value < 0) throw new Exception();
        preco = value;
    }
}
```

Static

```
public static class Util
{
    public static void Metodo() { }
}
Util.Metodo(); // sem 'new'
```

Construtor

```
public Produto(string nome) : this(nome, 0)
{
}
```

Herança

```
public class Aluno : Pessoa
{
    public Aluno(string nome) : base(nome) { }
}
```

Abstrata

```
public abstract class Forma
{
    public abstract double CalcularArea();
}
```

Interface

```
public interface INotificador
{
    void Enviar(string msg);
}

public class Email : INotificador, ILogger
{
    public void Enviar(string msg) { }
    public void Log(string msg) { }
}
```

MATERIAL DE APOIO

Livros Recomendados

- **C# 12 and .NET 8** - Andrew Troelsen
- **Clean Code** - Robert C. Martin
- **Head First Design Patterns** - Freeman & Freeman

Links Úteis

- Microsoft Learn: <https://learn.microsoft.com/pt-br/dotnet/csharp/>
- C# Documentation: <https://docs.microsoft.com/pt-br/dotnet/csharp/>
- LINQ Tutorial: <https://www.tutorialsteacher.com/linq>

Bons estudos! 📖✨

Elaborado por: Prof. Geovani

Instituição: Faculdade Einstein de Limeira

Disciplina: Desenvolvimento Web Back-End

Versão: 1.0 - Revisão Semestral 2026